Smart Patio Shield - Motion Test (Notebook 07)

Question: does giving the CNN *temporal context* (consecutive hourly frames, so it can see the cloud field moving) improve Model 2, and if so, does that flow through to improve fusion (Model 3)?

Method: stack N consecutive hourly patches as channels and train the same ResNet-18. Compare n_frames = 1 (single frame, the current Model 2), 2, and 3.

Uses only patches already on disk: no new download. Frames are hourly, which is coarse for storm motion, so this is a *conservative* test: if even hourly stacking helps, finer sub-hourly stacking (future work) would likely help more; if it doesn't, single-frame is near the ceiling.

Fairness control: n=2 and n=3 can only form samples where 1–2 hours of history exist, so they see fewer samples than n=1. To compare like-for-like, **all configs are trained and evaluated on the intersection of samples that all three can form.** Otherwise n=1 would get easy early-hour samples the others don't, biasing the comparison.

Sections:

- **0** Setup (same idempotent setup as notebook 05)
- **1** Build the common sample set across n=1,2,3
- **2** Train each config, evaluate on the shared test set
- **3** If motion helps: regenerate the vision branch and re-run fusion
- **4** Save results

Section 0 - Session setup

Same as notebook 05. Requires `src/models/goes_temporal.py` to be pushed to the repo.

In [ ]:
```
import os, torch
print("GPU:", torch.cuda.get_device_name(0) if torch.cuda.is_available()
      else "NONE — Runtime > Change runtime type > T4 GPU")
if not os.path.exists("/content/smart-patio-shield"):
```

```
    from getpass import getpass
    tok = getpass("GitHub token: ")
    !git clone https://{tok}@github.com/romayneg/smart-patio-shield.git /content/smart-patio-shield
%cd /content/smart-patio-shield
!git pull -q
print("code up to date")
```

In [ ]:

```
from google.colab import drive
drive.mount('/content/drive')
DRIVE = "/content/drive/MyDrive/smart-patio-shield"

import shutil
def relink(target, linkname):
    if os.path.islink(linkname):   os.unlink(linkname)
    elif os.path.isdir(linkname):  shutil.rmtree(linkname)
    elif os.path.exists(linkname): os.remove(linkname)
    parent = os.path.dirname(linkname)
    if parent: os.makedirs(parent, exist_ok=True)
    os.symlink(target, linkname)

relink(f"{DRIVE}/data/goes",                   "data/raw/goes")
relink(f"{DRIVE}/data/image_labels.parquet",   "data/processed/image_labels.parquet")
relink(f"{DRIVE}/data/patio_features.parquet","data/processed/patio_features.parquet")
relink(f"{DRIVE}/models",                       "models")
print("day-files:", len(os.listdir("data/raw/goes")))
assert os.path.exists("src/models/goes_temporal.py"), \
    "goes_temporal.py not found — commit & push it, then re-run Section 0."
print("setup complete")
```

Section 1 - Build the common sample set

We build the n=3 dataset first: its keys are exactly the hours that have 2 hours of prior history, i.e. the strictest set. Every config is then restricted to **these same keys** so n=1, n=2, n=3 are compared on identical samples.

I use **IR_ONLY (C13+C09)** for the motion test, on purpose:

- it isolates the *motion* question from the separate C02 day/night confound the review raised;
- it's lighter (fewer channels → faster, and 3 frames of IR = 6 channels vs 9 with visible).

If motion clearly helps on IR, adding visible back is a follow-up.

In [ ]:
```
import importlib, gc
import src.models.goes_temporal as gt
import src.models.cnn as cnn
importlib.reload(gt); importlib.reload(cnn)

patches = gt._load if False else None  # (patches come from goes_dataset loader)
import src.models.goes_dataset as gd
importlib.reload(gd)
patches = gd._load_all_patches()
print(f"patches in memory: {len(patches):,}")

BANDS = gt.IR_ONLY   # motion test on IR only
MAXF  = 3            # strictest history requirement defines the common set

def build(split, n_frames, norm=None, restrict_keys=None):
    ds = gt.GoesTemporalDataset(split, channels=BANDS, n_frames=n_frames,
                                norm_stats=norm, patches=patches)
    if restrict_keys is not None:
        keep = set(restrict_keys)
        ds.keys = [k for k in ds.keys if k in keep]
    return ds

# Strictest (n=3) keys per split define the common sample set
common = {}
for split in ["train", "val", "test"]:
    d3 = build(split, MAXF)
    common[split] = set(d3.keys)
    print(f"{split}: common (n=3-eligible) samples = {len(common[split]):,}")
```

Section 2 - Train n=1, 2, 3 on the shared samples

For each config: build train/val/test restricted to the common keys, compute TRAIN-only norm stats, train the CNN (`in_channels = len(BANDS) * n_frames`), and record the best validation PR-AUC and the held-out test PR-AUC.

Same training recipe as Model 2 (class-weighted BCE, Adam lr=1e-4, PR-AUC early stopping) so the only thing changing is the number of frames.

In [ ]:
```
from src.models.cnn import train_model, evaluate
from torch.utils.data import DataLoader
from sklearn.metrics import average_precision_score
import numpy as np, torch

results_motion = {}

for nf in [1, 2, 3]:
    print(f"\n{'='*30} n_frames = {nf} {'='*30}")
    train_ds = build("train", nf, restrict_keys=common["train"])
    NORM = train_ds.norm_stats
    val_ds  = build("val",  nf, norm=NORM, restrict_keys=common["val"])
    test_ds = build("test", nf, norm=NORM, restrict_keys=common["test"])
    in_ch = len(BANDS) * nf
    print(f"samples train/val/test: {len(train_ds)}/{len(val_ds)}/{len(test_ds)}  in_channels={in_ch}")

    model, history = train_model(train_ds, val_ds, in_channels=in_ch,
                                 epochs=30, batch_size=64, lr=1e-4, patience=5)

    # held-out test PR-AUC
    dev = "cuda" if torch.cuda.is_available() else "cpu"
    test_loader = DataLoader(test_ds, batch_size=128, shuffle=False, num_workers=2)
    ev = evaluate(model, test_loader, dev)
    best_val = max(h["val_pr_auc"] for h in history)
    results_motion[nf] = {"val_pr_auc": round(best_val,4),
                          "test_pr_auc": round(ev["pr_auc"],4),
                          "n_train": len(train_ds), "in_channels": in_ch}
    # keep the models/probs we may need for fusion
    results_motion[nf]["_model"] = model
    results_motion[nf]["_test_ds"] = test_ds
    print(f"n={nf}: val {best_val:.4f}  test {ev['pr_auc']:.4f}")

print("\n\n==== MOTION TEST SUMMARY (shared samples, IR-only) ====")
print(f"{'frames':>7} {'in_ch':>6} {'val PR-AUC':>11} {'test PR-AUC':>12}")
for nf in [1,2,3]:
    r = results_motion[nf]
    print(f"{nf:>7} {r['in_channels']:>6} {r['val_pr_auc']:>11} {r['test_pr_auc']:>12}")
print("\nReference: single-frame IR+visible Model 2 test PR-AUC = 0.4397")
print("(n=1 here is IR-only on the restricted common set, so it won't equal 0.44 exactly.)")
```

How to read this:

- **Monotonic lift 1→2→3** (test PR-AUC rising with frames) = motion genuinely helps → proceed to Section 3 and push it through fusion.
- **Flat or noisy** (differences within ~±0.01, the run-to-run noise you measured earlier) = hourly temporal context doesn't help this task → document as a tested negative, single-frame is near ceiling.
- **n=1 vs the 0.44 reference:** n=1 here is IR-only on a *smaller* common sample set, so expect it near the IR-only 0.427 you already have, not 0.44. What matters is the *trend across frames*, measured on identical samples.

Section 3 - If motion helps: push the best config through fusion

In [ ]:
```
# ---- set which config to promote ----
BEST_NF = max([1,2,3], key=lambda k: results_motion[k]["test_pr_auc"])
print("best n_frames by test PR-AUC:", BEST_NF)

if results_motion[BEST_NF]["test_pr_auc"] - results_motion[1]["test_pr_auc"] < 0.01:
    print("No meaningful motion lift (<0.01). Skipping fusion re-run; document as tested negative.")
else:
    import src.models.fusion as fus
    importlib.reload(fus)
    import numpy as np, pandas as pd, torch
    from torch.utils.data import DataLoader

    model = results_motion[BEST_NF]["_model"]
    # vision branch predictions on val+test for the common samples
    def vision_probs(model, ds):
        dev="cuda" if torch.cuda.is_available() else "cpu"
        model=model.to(dev).eval()
        loader=DataLoader(ds,batch_size=128,shuffle=False)
        ps,ys=[],[]
        with torch.no_grad():
            for x,y in loader:
                ps.append(torch.sigmoid(model(x.to(dev)).squeeze(1)).cpu().numpy()); ys.append(y.numpy())
        return pd.DataFrame({"key":ds.keys,"p_img":np.concatenate(ps),"y":np.concatenate(ys)})

    val_ds  = build("val",  BEST_NF, norm=results_motion[BEST_NF]["_test_ds"].norm_stats, restrict_keys=common["val"]
    img_val  = vision_probs(model, val_ds)
    img_test = vision_probs(model, results_motion[BEST_NF]["_test_ds"])

    tab = fus.tabular_branch("models/xgboost_baseline.json","models/baseline_training.manifest.json")
```

```
    # align on key
    def align_simple(tab_frame, img_frame):
        m = tab_frame.merge(img_frame[["key","p_img"]], on="key", how="inner")
        return {"y":m["y"].to_numpy() if "y" in m else img_frame.set_index("key").loc[m["key"],"y"].to_numpy(),
                "p_tab":m["p_tab"].to_numpy(),"p_img":m["p_img"].to_numpy()}
    val_al  = align_simple(tab["val"][0],  img_val)
    test_al = align_simple(tab["test"][0], img_test)

    p_late,_ = fus.late_fusion(val_al, test_al)
    print("=== Fusion with temporal Model 2 ===")
    fus.report("Model 1 (tabular)", test_al["y"], test_al["p_tab"])
    fus.report(f"Model 2 temporal (n={BEST_NF})", test_al["y"], test_al["p_img"])
    fus.report("Late fusion (temporal vision)", test_al["y"], p_late)
```

Section 4 — Save motion-test results

In [ ]:

```
import json
from datetime import datetime, timezone

payload = {
    "created_utc": datetime.now(timezone.utc).isoformat(),
    "experiment": "temporal motion test — stack N consecutive hourly IR frames",
    "bands": gt.IR_ONLY,
    "fairness": "all configs trained/evaluated on the n=3-eligible common sample set",
    "reference_single_frame_ir_visible": 0.4397,
    "results": {str(nf): {k:v for k,v in results_motion[nf].items() if not k.startswith("_")}
                for nf in [1,2,3]},
}
out = f"{DRIVE}/models/motion_test_results.json"
json.dump(payload, open(out,"w"), indent=2)
print("saved:", out)
print(json.dumps(payload["results"], indent=2))
```